

10.3.2025.

Module-3 (Choice)

Syntax Analysis

⇒ LR parsing:

1. SLR (Simple LR)
2. CLR (Canonical LR)
3. LALR (Lookahead LR)

1) Construct a simple LR parser (SLR) for the given grammar. $E \rightarrow BB \rightarrow \text{rule 1}$

 $B \rightarrow cB \rightarrow \text{rule 2}$ $B \rightarrow d \rightarrow \text{rule 3}$

a) Construct LR(0) automaton.

b) Build SLR parsing table.

c) Parse the given string. String: ccd $\bar{d}$.

Ans: a) Step 1: Find the augmented grammar.

 $\bar{E} \rightarrow E$ $\bar{E} \rightarrow BB$ $\bar{B} \rightarrow cB$ $\bar{B} \rightarrow d$

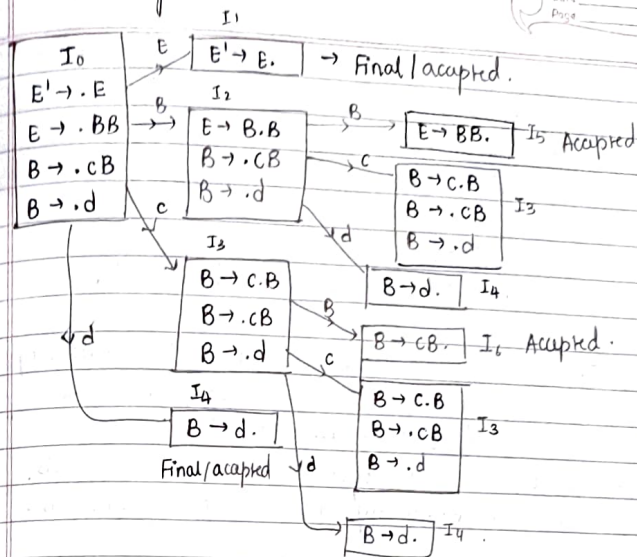
Step 2: Closure set/items:

 $\bar{E} \rightarrow \cdot E$ $\bar{E} \rightarrow \cdot BB$ $\bar{B} \rightarrow \cdot cB$ $\bar{B} \rightarrow \cdot d$

Step 3: GOTO function: $\text{GOTO}(\bar{I}_0, X)$ where $X = \{E, B, c, d\}$

Step 4: Automaton $\rightarrow \text{LR}(0)$

After the non-terminal then write that particular production also. If it is terminal then neglect it.

 $\text{GOTO}(\bar{I}_1, X) \rightarrow \text{nothing} / \text{Hence accepted.}$ $\text{GOTO}(\bar{I}_2, X)$ where $X = \{B, c, d\}$ $\text{GOTO}(\bar{I}_3, X)$ where $X = \{B, c, d\}$ $\text{GOTO}(\bar{I}_4, X) \rightarrow \text{Accepted.}$ $\text{GOTO}(\bar{I}_5, X) \rightarrow \text{Accepted.}$ $\text{GOTO}(\bar{I}_6, X) \rightarrow \text{Accepted.}$

b)	State	Action			GOTO	
		c	d	\$	E	B
	$\bar{I}_0$	S3	S4		1	2
	$\bar{I}_1$			Accept.		
	$\bar{I}_2$	S3	S4		5	5
	$\bar{I}_3$	S3	S4			6
	$\bar{I}_4$	r3	r3	r3		
	$\bar{I}_5$			r1		
	$\bar{I}_6$	r2	r2	r2.		

For action, write terminals. Hence find first and follow.

First (E) $\rightarrow \{c, d\}$

First (B) $\rightarrow \{c, d\}$.

Follow (E) $\rightarrow \{\$ \}$.

Follow (B) $\rightarrow \{\text{first}(B) \cup \text{follow}(E)\} = \{c, d, \$ \}$.

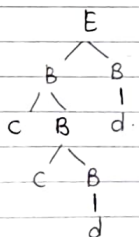
→ If it is accepted state, then look for follow of that production. If it is only \$ then write ACCEPT. ((E $\rightarrow$ E) ~~is~~ rule ill'a)

→ If follow contains ~~{c, d, \$}~~, other non-terminals then look which rule produces that production.

→ If we have E, then only write accept.

c) Stack	Input	Refu table (parse) Action
\$0 $\rightarrow$ By default	ccdd\$	Shift c $\rightarrow$ S3.
\$0c3	cdd\$	Shift c $\rightarrow$ S3.
\$0c3c3	dd\$	Shift c $\rightarrow$ S4.
\$0c3c3d4	d\$	Reduce B $\rightarrow$ d.
\$0c3c3B6	d\$	Reduce B $\rightarrow$ CB.
\$0c3B6	d\$	Reduce B $\rightarrow$ CB.
\$0B2	d\$	Shift d $\rightarrow$ S4.
\$0B2d4	\$	Reduce B $\rightarrow$ d.
\$0B2B5	\$	Reduce E $\rightarrow$ BB.
\$0E1	\$	Accept.

Refer
I3 of B $\leftarrow$
if there is
no number



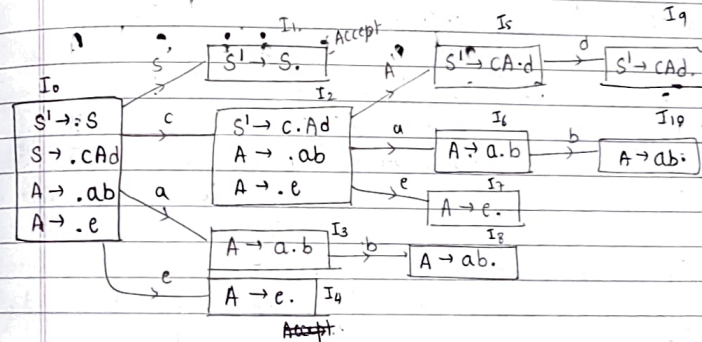
→ Parse tree

2) $S \rightarrow cAd$ String: ccd\$.
 $A \rightarrow ab$
 $A \rightarrow e$

Augmented grammar: $S' \rightarrow S$
 $S \rightarrow cAd$
 $A \rightarrow ab$
 $A \rightarrow e$

closure: $S' \rightarrow \cdot S$
 $S \rightarrow \cdot cAd$
 $A \rightarrow \cdot ab$
 $A \rightarrow \cdot e$

GOTO (I_0, X), $X = \{S, c, a, e\}$.



GOTO (I_1, X) $\rightarrow$ Accept.
GOTO (I_2, X) - $X = \{A, a, e\}$.
GOTO (I_3, X) - $X = \{b\}$
GOTO (I_4, X) - Accept
GOTO (I_5, X) - $X = \{d\}$
GOTO (I_6, X) - $X = \{b\}$

first(S) $\rightarrow \{c\}$
first(A) $\rightarrow \{a, e\}$
follow(S) $\rightarrow \{\$ \}$
follow(A) $\rightarrow \{d\}$

State	Action					Goal	
b	c	a	e	d	\$	s	A
I ₀	S2	S3	S4				
I ₁					Accept		
I ₂		S6	S7				
I ₃							
I ₄							
I ₅							
I ₆							
I ₇							
I ₈							
I ₉							
I ₁₀							

State	Action			Goto				
	c	a	e	\$	d	b	s	A
I ₀	S2	S3	S4				1	
I ₁				Accept				0
I ₂		S3	S4					5
I ₃						S6		
I ₄					r3			
I ₅				m	S7			
I ₆					r2			
I ₇				r1				

3. $S \rightarrow DD$ String: aadd\$
 $D \rightarrow ad$
 $D \rightarrow d$

$$\gamma_1: S \rightarrow AS$$
$$\gamma_2: S \rightarrow b$$

73. $A \rightarrow SA$

~~74. $A \rightarrow \alpha$~~

3) $S \rightarrow AS / b$
 $A \rightarrow SA / a$

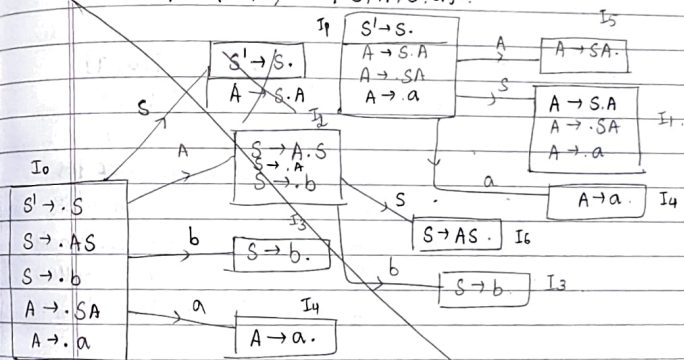
1) construct LR(0) Automaton

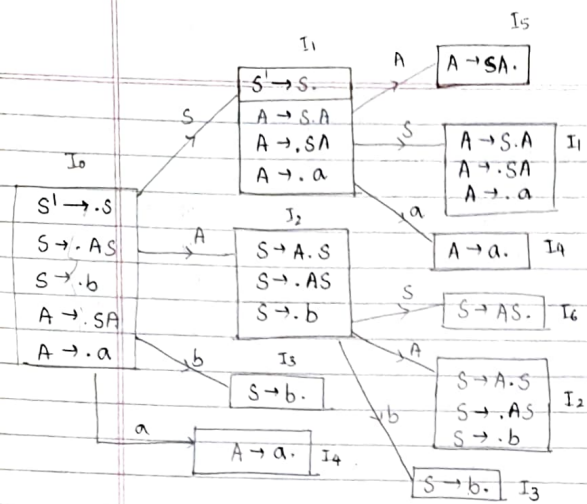
2) Parse the given string 'abab'

Augmented grammars : $S' \rightarrow S$

$$S \rightarrow AS/b$$
$$A \rightarrow SA / a$$
$$g' \rightarrow s$$
$$S \rightarrow AS$$
$$S \rightarrow b$$
 ~~$A \rightarrow SA$~~
$$\underline{A \rightarrow a}$$

closure : $S' \rightarrow \cdot S$

$$S \rightarrow \cdot AS$$
$$S \rightarrow \cdot b$$
$$A \rightarrow .SA$$
$$A \rightarrow \cdot a$$
$$\text{GOTO}(I_0, x) = \{S, A, b, a\}$$

$$\text{GOTO}(I_1, X) = \{A, s, a\}.$$
$$\text{GOTO}(\mathbb{I}_2, \lambda) = \{s, b\}.$$
$$\text{GOTO}(I_0, X) = \{S, A, b, a\}.$$



$GOTO(I_1, X) = \{A, S, a\}$
 $GOTO(I_2, X) = \{S, A, b\}$

$first(S) \rightarrow \{b, a\}$
 $first(A) \rightarrow \{a, b\}$
 $follow(S) \rightarrow \{\$, b, a\}$
 $follow(A) \rightarrow \{a, b\}$

State	Action			GOTO	
	a	b	\$	S	A
I ₀	S4	S3		1	2
I ₁	S4		Accept	1	5
I ₂		S3		6	2
I ₃	r2	r2	r2		
I ₄	r4	r4			
I ₅	r3	r3			
I ₆	r1	r1	r1		

To write look for rule no then write it for all terminals by looking at follow of that production.

Stack	I/p	Action
\$0	abab\$	Shift: a → S4
\$0a4	bab\$	Reduce A → a
\$0a2	bab\$	Shift: b → S3
\$0a2b3	ab\$	Reduce S → b

Refer I₀ of A in GOTO

\$0a2S6 ab\$ Reduce $S \rightarrow AS$

\$0S1 ab\$ Shift: a → S4

\$0S1a4 b\$ Reduce A → a

\$0S1A5 b\$ Reduce A → SA

\$0A2 b\$ Shift b → S3

\$0A2b3 \$ Reduce S → b

\$0A2S6 \$ Reduce S → AS

\$0S1 \$ Accept

⇒ Syntax-directed translation:
 SDD: non-terminal - val
 non-terminal - lexical

1. Write the SDD for the following grammar. (B-U)

$S \rightarrow E_n$ Here n - end of file.
 $E \rightarrow E + T / T$
 $T \rightarrow T * F / F$
 $F \rightarrow (E) / \text{digit}$

Construct annotated parse tree for the input string 5*6+7.

$S \rightarrow T_n$
 $T \rightarrow T * F / F$
 $F \rightarrow \text{digit}$

Production	Semantic Rules
$S \rightarrow T_n$	$S.val \Rightarrow T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val * F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

Don't need.

~~$S \rightarrow E_n$
 $E \rightarrow E + T / T$
 $F \rightarrow (E)$~~

Final SDD :

Production P

$S \rightarrow E_n$

$E \rightarrow E_1 + T$

$E \rightarrow T$

$T \rightarrow T_1 * F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow \text{digit}$

Semantic Rules

$S.val = E.val$

$E.val = E_1.val + T.val$

$E.val = T.val$

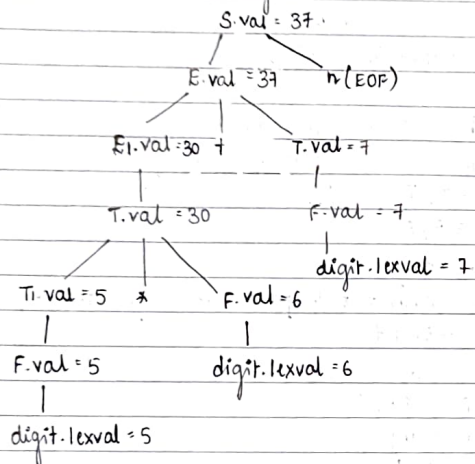
$T.val = T_1.val * F.val$

$T.val = F.val$

$F.val = E.val$

$F.val = \text{digit.lexval}$

Parse tree : String : $5 * 6 + 7$



2. Write a SDD for a simple desk calculator & show annotated parse tree for the given input $(3+4) * (5+6)$ (B-U)

$S \rightarrow E_n$

$E \rightarrow E_1 + T / T / E - T$

$T \rightarrow T_1 * F / F / T / F$

$F \rightarrow (E) / \text{digit}$

Production P

$S \rightarrow E_n$

$E \rightarrow E_1 + T$

$E \rightarrow T$

$E \rightarrow E_1 - T$

$T \rightarrow T_1 * F$

$T \rightarrow F$

$T \rightarrow T_1 / F$

$F \rightarrow (E)$

$F \rightarrow \text{digit}$

Semantic Rules

$S.val = E.val$

$E.val = E_1.val + T.val$

$E.val = T.val$

$E.val = E_1.val - T.val$

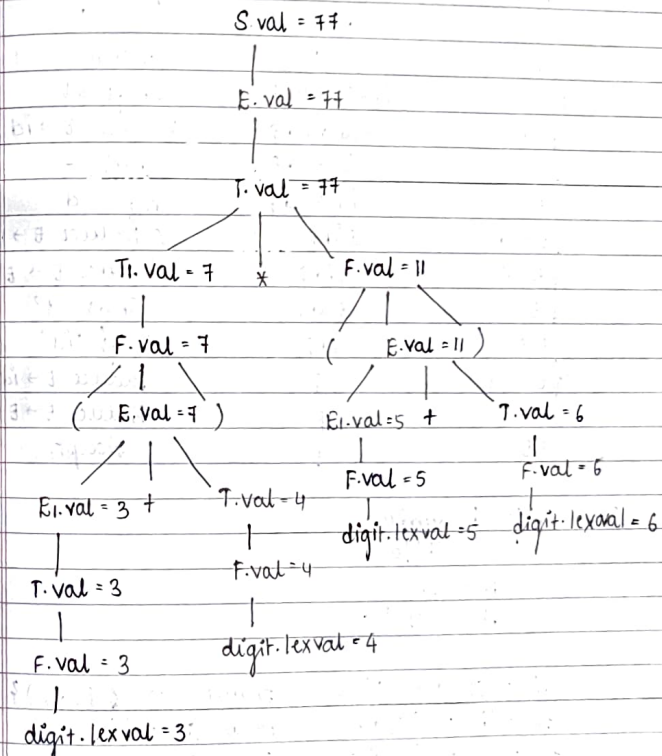
$T.val = T_1.val * F.val$

$T.val = F.val$

$T.val = T_1.val / F.val$

$F.val = E.val$

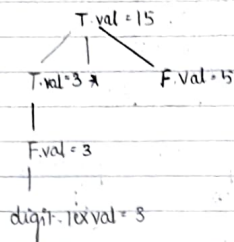
$F.val = \text{digit.lexval}$



$$T \rightarrow T * F \mid F$$

1. $T \rightarrow T * F \mid F$

$F \rightarrow \text{digit}$



Production P

$$T \rightarrow T * F$$
$$F \rightarrow F$$

$F \rightarrow \text{digit}$

Semantic Rules.

$$T.val = T.val * F.val$$
$$T.val = F.val$$
$$f.val = digit.lexval.$$

2. Obtain SDD for the following grammar using Top-down approach for the exp $2*3$.

$$T \rightarrow T * F / F$$

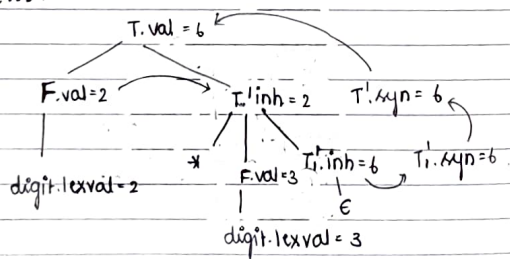
$f \rightarrow \text{digit}$

Left recursion:

$$T \rightarrow FT^{-1}$$
$$T' \rightarrow *FT'_1 / \epsilon$$

$F \rightarrow \text{digit}$.

Parse tree:



SDD :

$$T'.inh = F.val$$

T. val = T'. μ_{yr}

$$T_1' \cdot inh = T' \cdot inh * F. val$$
$$\left(\begin{array}{c} T_i \cdot \text{syn} \\ \downarrow \\ T_i \cdot \text{inh} \end{array} \right) = T_i \cdot \text{inh} \quad \leftarrow$$
$$\downarrow T'_{syn} = T_{syn}$$
$$F.val = digit.lexval$$

3. $S \rightarrow E_n$

$$E \rightarrow E + T / T$$
$$T \rightarrow T * F / F$$
$$F \rightarrow (E) / \text{digit}$$

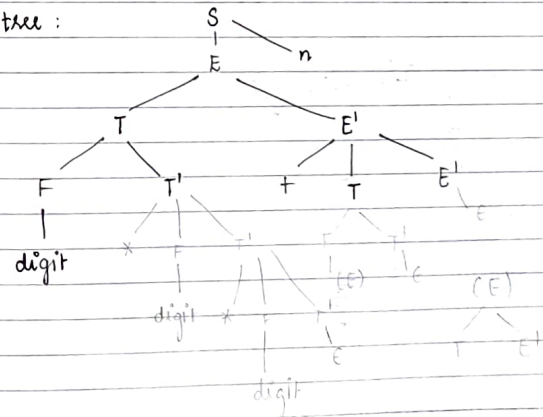
Top-down approach.

I/P : $1 * 2 * 3 + (4 + 5)$

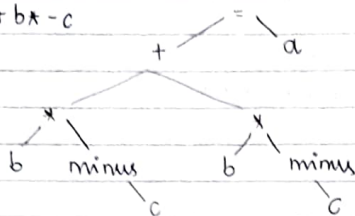
Left recursion :

$$S \rightarrow E_n$$
$$E \rightarrow TE'$$
$$E' \rightarrow +TE_1' / E$$
$$T \rightarrow FT'$$
$$T' \rightarrow *FT_1' / \epsilon$$
$$F \rightarrow (E) / \text{digit}.$$

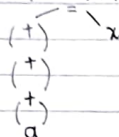
Parse tree :



4. $a = b * -c + b * -c$



5. $x = ((a+a) + (a+a)) + ((a+a) + (a+a))$



⇒ SDD:

Note: Node (op, left, right)

 $E \rightarrow E_1 + T$ E.node = ~~new~~Node ('+', E_1.node, T.node) $\rightarrow F \rightarrow id$

F.node = newleaf (id, id.entry)

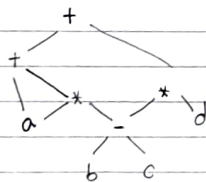
 $\rightarrow F \rightarrow a$

F.node = newleaf (id, entry-a)

 $\rightarrow F \rightarrow num$

F.node = newleaf (num, num.val)

$a + a * (b - c) + (b - c) * d$



1) Write SDD for this directed acyclic graph.

SDD:

 $p_1 = \text{newleaf} (id, \text{entry}-a)$ $p_2 = \text{newleaf} (id, \text{entry}-b)$ $p_3 = \text{newleaf} (id, \text{entry}-c)$ $p_4 = \text{newleaf} (id, \text{entry}-d)$ $p_5 = \text{Node} ('-', p_2, p_3)$ $p_6 = \text{Node} ('*', p_1, p_5)$ $p_7 = \text{newleaf} (id, \text{entry}-a) = p_1$ $p_8 = \text{newleaf} ('+', p_1, p_6)$ $p_9 = \text{newleaf} (id, \text{entry}-b) = p_2$ $p_{10} = \text{newleaf} (id, \text{entry}-c) = p_3$ $p_{11} = \text{Node} ('-', p_2, p_3) = p_5$ $p_{12} = \text{Node} ('*', p_5, p_4)$ $p_{13} = \text{Node} ('+', p_8, p_{12})$

Imp ⇒ Three address code:

1. Quadruples: (op, arg1, arg2, result)

2. Triples: (op, arg1, arg2)

3. Indirect triples: makes use of pointers.

 $x + y * z$ (TAC) $t_1 = x + y$ $t_2 = t_1 * z$

BODMAS

↓
unary (minus)

Quadruples:

1. $a = b * -c + b * -c$ $t_1 = \text{minus } c$ $t_2 = b * t_1$ $t_3 = \text{minus } c$ $t_4 = b * t_3$ $t_5 = t_2 + t_4$ $a = t_5$

Pos	op	arg1	arg2	result
0	minus	c		t1
1	*	b	t1	t2
2	minus	c		t3
3	*	b	t3	t4
4	+	t2	t4	t5
5	=	t5		a

Triples

pos	op	arg 1	arg 2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	(4) a	(4)

operands - arg 1
reference - arg 2

Indirect triples:

#	Statement
(0)	(52)
(1)	(53)
(2)	(54)
(3)	(55)
(4)	(56)
(5)	(57)

any random number
but continuous
other than position.

pos	op	arg 1	arg 2
0	minus	c	
1	*	b	(52)
2	minus	c	
3	*	b	(54)
4	+	(53)	(55)
5	=	a	(56)

$$2. (x+y) * (y+z) + (x+y+z)$$

$$t1 = x+y$$

$$t2 = y+z$$

$$\therefore t3 = t1 * t2$$

$$t4 = t3 + t2$$

$$t5 = t3 + t4$$

Quadruples

pos	op	arg 1	arg 2	result
0	+	x	y	t1
1	+	y	z	t2
2	*	t1	t2	t3
3	+	t1	z	t4
4	+	t3	t4	t5

Triples:

pos	op	arg 1	arg 2
0	+	x	y
1	+	y	z
2	*	(0)	(1)
3	+	(0)	z
4	+	(2)	(3)

Indirect triples:

#	Statement	pos	op	arg 1	arg 2
(0)	(52)	0	+	x	y
(1)	(53)	1	+	y	z
(2)	(54)	2	*	(52)	(53)
(3)	(55)	3	+	(52)	z
(4)	(56)	4	+	(54)	(55)